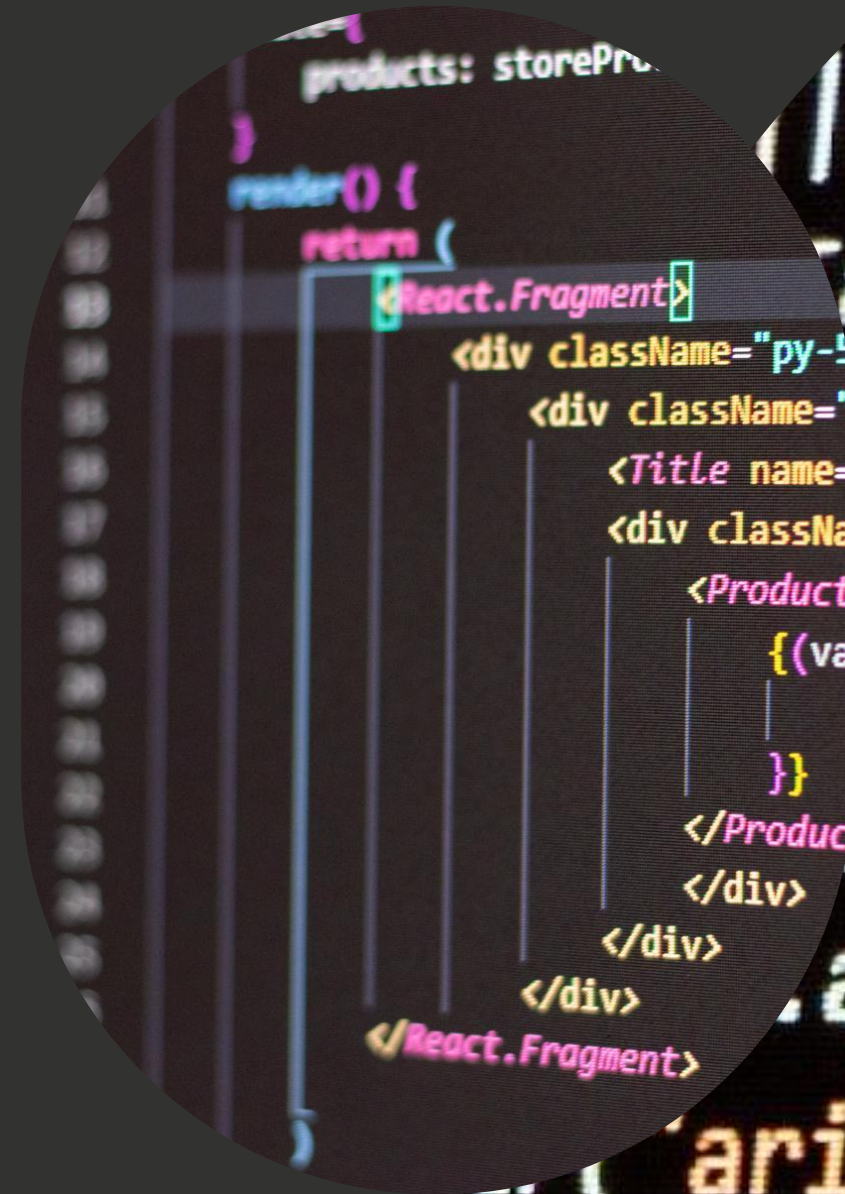


Vehicle Management (KalapujSolU0)

Proyecto POO desarrollado en CSharp

Sol Ailen Kalapuj

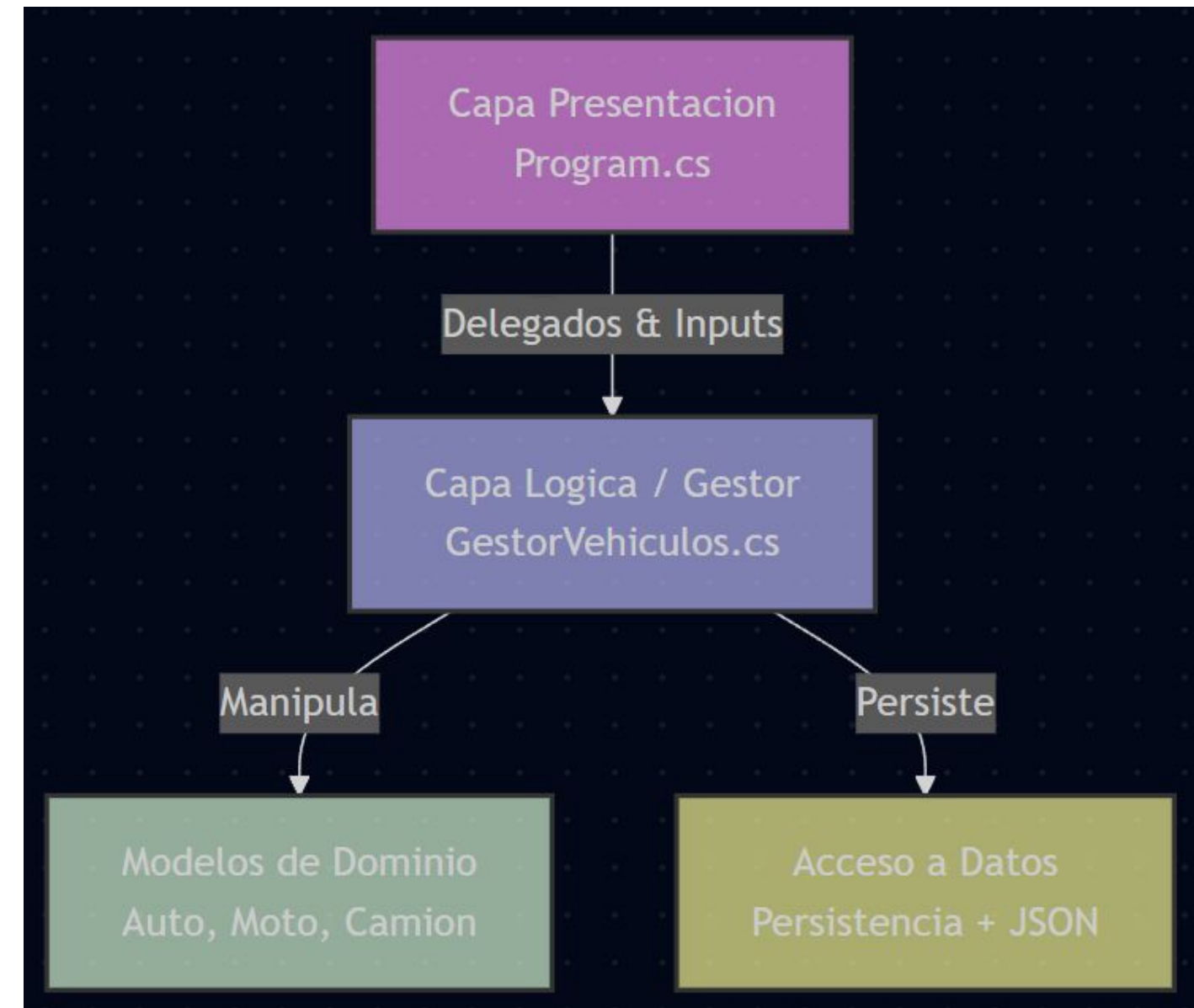
Programación orientada a objetos - UCES



```
(b));c.VERSION="3.3  
|(d=b.attr("href"),d=  
Target:b[0]}),g=a.Eve  
e(b.closest("li"),c  
t:e[0]}))}}}},c.prot  
)find('[data-toggle  
addClass("in")):b.re  
aria-expanded",!0),e&&  
;g.length&&h?g.one("bsTrans  
tab.Constructor=c,a.fn.tab.  
.tab.data-api",'[data-togg  
this.each(function(){var d
```


Arquitectura del Sistema

- 1 Separación estricta de responsabilidades (SoC).
- 2 Capa IU: Manejo de entrada/salida y validación de usuario.
- 3 Capa de Negocio: Organización de lógica y reglas de negocio.
- 4 Capa de Datos: Modelos completos y persistencia en JSON.



Diseño del Dominio: Herencia y Validación



Abstracción:

Clase base Vehiculo (No instanciable).

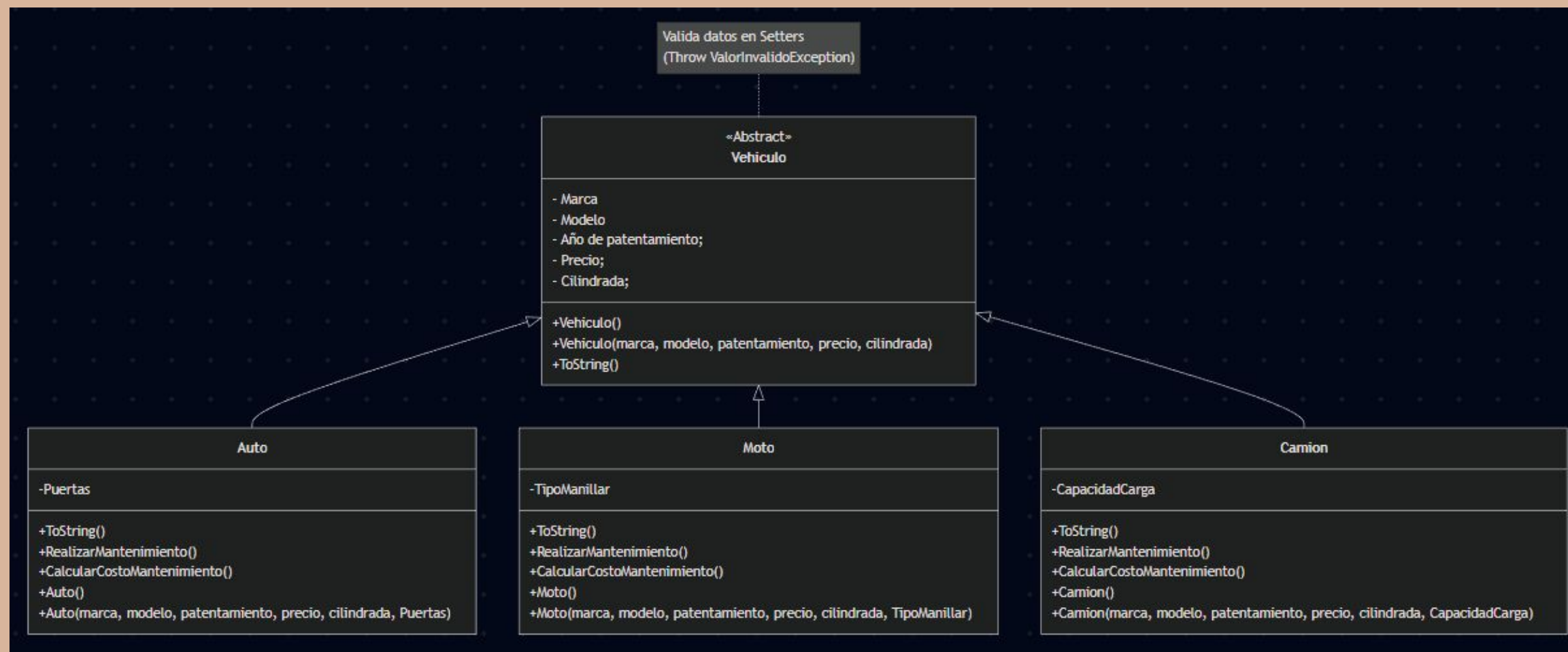
Herencia:

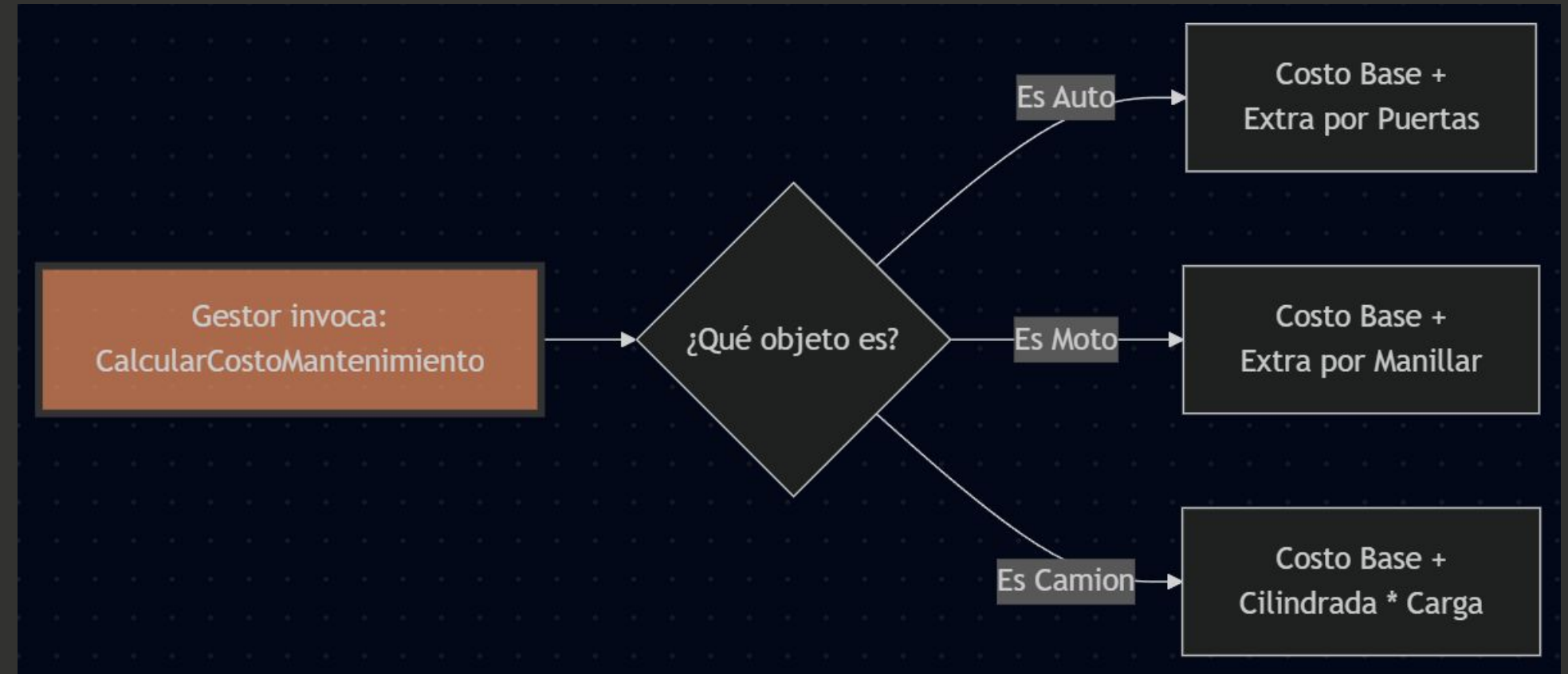
Auto, Moto, Camion extienden la funcionalidad base.

Encapsulamiento:

Validaciones integradas en Properties.

Ejemplo: El año de patentamiento o precio negativo lanzan excepciones inmediatas.





Polimorfismo: Flexibilidad de Comportamiento

Uso de Interfaces:
IMantenimiento

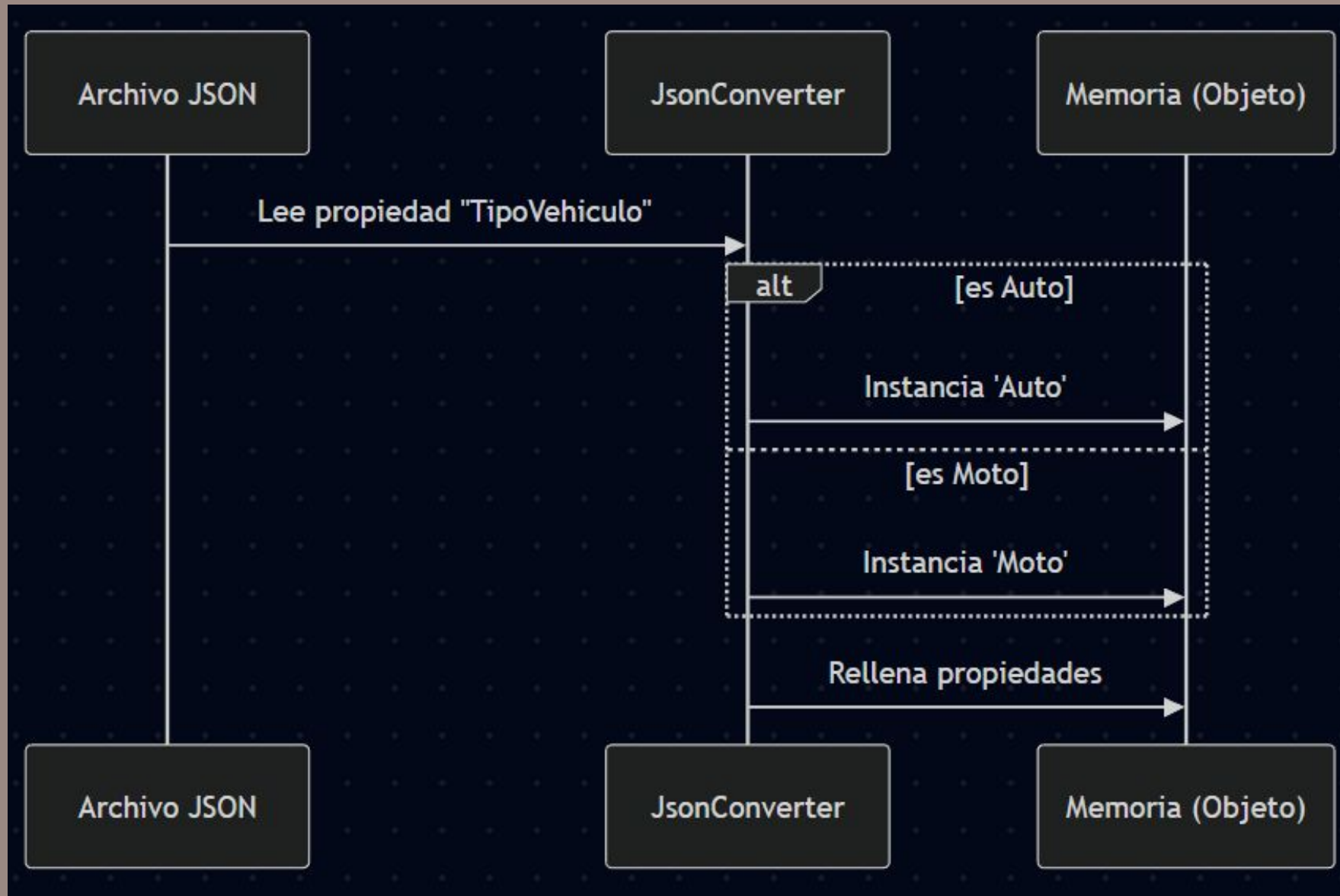
Desacoplamiento:

El Gestor no necesita conocer el tipo específico.

Cálculo Dinámico:

- Moto: Costo base + Manillar.
- Camión: Costo base + Carga
- Cilindrada

Ingeniería de Software: Persistencia y Errores



Desafío JSON:

Pérdida de tipos al serializar listas heterogéneas.

Solución:

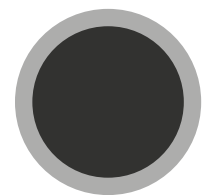
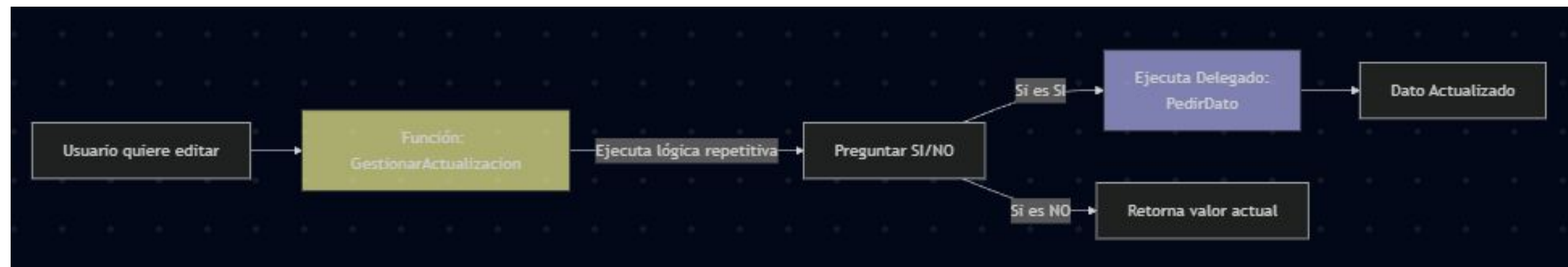
VehiculoJsonConverter (Polimorfismo en serialización).

Robustez:

Manejo centralizado de errores con `ValorInvalidoException`.

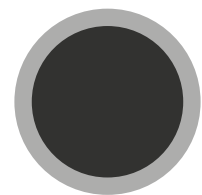
- Evita el cierre inesperado de la aplicación ("Crash").

Clean Code: Patrones en la Interfaz



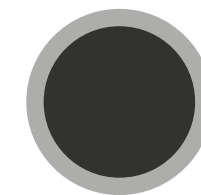
DRY (Don't Repeat Yourself):

Eliminación de código repetitivo.



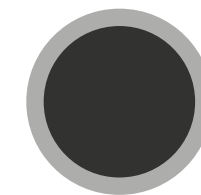
Delegados (Func/Action):

Funciones pasadas como parámetros.



Método GestionarActualizacion:

Encapsula la lógica de pregunta/respuesta.



Resultados:

Código mantenible, legible y escalable.

Evolución del Diseño: Patrones y Desacoplamiento

Patrón Builder

Problema actual:

Lógica repetitiva y dispersa en el Gestor (if / operador ternario) para validar actualizaciones.

Solución:

Encapsular la regla "Si es vacío, ignóralo" dentro de una clase VehiculoBuilder.

Resultado:

Código fluido (.ConMarca("X").ConPrecio(100)) y validación centralizada.

Desacoplamiento de Servicios

Problema:

El modelo Moto depende directamente de la clase estática CostoManillar.

Solución:

Inyección de Dependencias. Los modelos de datos no deben depender de lógica externa.

Conclusiones



Más que código

La POO no es solo sintaxis, es una forma de modelar la realidad de manera robusta.

Integridad de Datos

Gracias al encapsulamiento y validaciones, el sistema protege la información de estados inválidos.

Escalabilidad

El uso de herencia e interfaces permite agregar nuevos vehículos sin romper el código existente.

Persistencia

El manejo de JSON y Streams transforma el programa en una aplicación útil y real.

¡Muchas
gracias!

